

Software Engineer — Step 6

Interview Problems

Roadmap objective: Prepare for technical interviews using structured problem solving

Level: Beginner → Intermediate

Last reviewed: September 2026

How to use these notes

Read the concepts first, reproduce the examples yourself, complete the practical lab, and answer the interview questions without looking at the notes. Use the official documentation links as the final authority for changing APIs and platform behavior.

Learning outcomes

- Clarify
- Brute force
- Optimize
- Complexity
- Edge cases
- Testing
- Communication
- Behavioral/project discussion

Core concepts

1. Interview performance improves when you communicate assumptions, constraints and trade-offs while solving.
2. Start with a correct baseline solution before optimizing.
3. Always state time and space complexity.
4. Use small examples to validate logic before coding the full solution.

Concept map

```
Interview Problems
|
+--> Learn the concept
|
+--> Reproduce a small example
|
+--> Apply it in a feature
|
+--> Test failure/edge cases
|
+--> Document the decision
```

Detailed study guide

1. Clarify

Study clarify through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in

a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

2. Brute force

Study brute force through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

3. Optimize

Study optimize through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

4. Complexity

Study complexity through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

5. Edge cases

Study edge cases through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

6. Testing

Study testing through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

Worked example

Interview solving template

1. Restate the problem.
2. Ask/confirm constraints.
3. Give a simple correct approach.
4. Analyze time and space.
5. Improve the approach.
6. Code clearly.
7. Test normal + edge cases.
8. Summarize trade-offs.

Practice variation: change one input, introduce one failure, predict the result, then run it. Rewrite the example from memory afterward.

Comparison table

Concept	Meaning	Practical use
Correctness	Produces right result	First priority

Concept	Meaning	Practical use
Optimization	Reduce cost	After baseline works
Communication	Explain reasoning	Critical in interviews

Common mistakes

- Jumping straight to code
- Not clarifying constraints
- No complexity analysis
- Skipping edge cases
- Silent problem solving

Best practices

- Keep responsibilities clear and small.
- Validate inputs at system boundaries.
- Prefer readable code over clever code.
- Test important success and failure paths.
- Use official documentation for current API/platform behavior.
- Keep secrets and credentials out of source control.
- Document important trade-offs so another developer can reproduce your reasoning.

Extended practical lab

Complete these tasks in order. The goal is to turn passive knowledge into evidence that you can actually use the topic.

Lab 1

- Solve one problem using brute force first.
- Write down what you expected, what happened, and why.

Lab 2

- State complexity.
- Write down what you expected, what happened, and why.

Lab 3

- Optimize it.
- Write down what you expected, what happened, and why.

Lab 4

- Test normal and edge cases.
- Write down what you expected, what happened, and why.

Lab 5

- Practice explaining while coding.
- Write down what you expected, what happened, and why.

Lab 6

- Review the solution and identify one improvement.
- Write down what you expected, what happened, and why.

Mini-project

Create a 30-question interview set across DSA, SQL, HTTP, OOP and system design. For each solution, write complexity and at least two edge cases.

Acceptance criteria

- A new developer can understand the project from its README.
- The main happy path works from start to finish.
- At least three edge/failure cases are tested.
- The project has meaningful Git commits.
- The project includes a short explanation of design choices.
- If deployment applies, production behavior is verified rather than assumed.

Interview preparation

Practice answering these aloud. A strong answer should define the concept, give a concrete example, mention one trade-off, and explain when you would use it.

Why start with brute force?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you optimize?

- Answer structure: definition → example → trade-off/limitation → practical use.

Why state complexity?

- Answer structure: definition → example → trade-off/limitation → practical use.

What edge cases matter?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you communicate while coding?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you recover from a mistake?

- Answer structure: definition → example → trade-off/limitation → practical use.

Debugging checklist

- Reproduce the problem consistently.
- Reduce it to the smallest failing example.
- Inspect inputs at each boundary.
- Check the first incorrect value, not only the final error.
- Read the complete error message and stack trace.
- Change one thing at a time.
- Retest the original failure after the fix.
- Add a regression test when the bug is important.

Glossary

Term	Your own definition	Example/use
Constraint	Write this in your own words.	Give one project example.
Brute force	Write this in your own words.	Give one project example.
Optimization	Write this in your own words.	Give one project example.
Complexity	Write this in your own words.	Give one project example.
Edge case	Write this in your own words.	Give one project example.
Invariant	Write this in your own words.	Give one project example.
Test case	Write this in your own words.	Give one project example.
Trade-off	Write this in your own words.	Give one project example.

Term	Your own definition	Example/use
Communication	Write this in your own words.	Give one project example.
Debugging	Write this in your own words.	Give one project example.

Self-test

- Explain Clarify without using its textbook definition.
- Show a small example of Brute force.
- What is the most important boundary in this topic?
- What is one failure mode and how would you diagnose it?
- What trade-off should a developer know before using this technique?
- How would you test the normal path and an edge case?
- How does this topic connect to the next roadmap step?
- What would you change if the system had 100× more users/data?
- What part of this topic would you explain differently to a beginner?
- What can you now build that you could not build before studying this step?

Final checklist

- I can explain and demonstrate Clarify.
- I can explain and demonstrate Brute force.
- I can explain and demonstrate Optimize.
- I can explain and demonstrate Complexity.
- I can explain and demonstrate Edge cases.
- I can explain and demonstrate Testing.
- I can explain and demonstrate Communication.
- I can explain and demonstrate Behavioral/project discussion.
- I completed the practical lab.
- I built or extended the mini-project.
- I tested at least three failure/edge cases.
- I can explain one trade-off.
- I can answer the interview questions without notes.
- I know where to find the authoritative documentation.

Recommended documentation

- <https://leetcode.com/>
- <https://www.geeksforgeeks.org/>

Recommended videos

- Software Engineering Interview Preparation (Existing DB resource 122)
- Top 6 Coding Interview Concepts - NeetCode (Existing DB resource 123)

Source note: resource references are based on the existing CareerCompass database plus verified web research. For changing APIs, versions, deployment settings and security guidance, consult the linked official documentation before production use.